Untitled

```python
from pathlib import Path
import pandas as pd
import geopandas as gpd
import altair as alt
import json

import warnings
warnings.filterwarnings('ignore')
alt.renderers.enable("png")
alt.data_transformers.disable_max_rows()

data_path = 'data/external'

# improve graph Resolution
import tempfile
from IPython.display import SVG, display, Image
import vl_convert as vlc

def display_altair_png(chart, scale=2):
    """
    Render an Altair chart to a PNG and display it inline.

    Parameters
    ----------
    chart : altair.Chart
        Altair chart object to render.
    scale : int, optional
        Resolution scaling factor for the PNG (default = 2). Use scale=2 for standard slides
    """
    png_bytes = vlc.vegalite_to_png(chart.to_dict(), scale=scale)

    # Write to a temporary PNG file and display
```

```
    with tempfile.NamedTemporaryFile(suffix=".png") as tmp:
        tmp.write(png_bytes)
        tmp.flush()
        display(Image(filename=tmp.name))
```

Part 0: Load and clean data

Loading data

```
# Set data path
DATA_DIR = Path("../data/derived-data")

# Crime
crime_raw = pd.read_csv(DATA_DIR / "Crimes_-_2001_to_Present_20260304.csv")
print("Crime loaded:", crime_raw.shape)

# Education
edu_raw = pd.read_csv(DATA_DIR / "ACSDT5Y2024.B15003-Data.csv")
print("Education loaded:", edu_raw.shape)

# Unemployment rate
unemp_raw = pd.read_csv(DATA_DIR / "ACSST5Y2024.S2301-Data.csv")
print("Unemployment loaded:", unemp_raw.shape)

# Income
income_raw = pd.read_csv(DATA_DIR / "ACSDT5Y2024.B19013-Data.csv")
print("Income loaded:", income_raw.shape)

# Population
pop_raw = pd.read_csv(DATA_DIR / "ACSDT5Y2024.B01003-Data.csv")
print("Population loaded:", pop_raw.shape)
pop_raw.head()

# Tract shapefile
tract_raw = gpd.read_file(DATA_DIR / "tl_2023_17_tract" / "tl_2023_17_tract.shp")
print("Tract loaded:", tract_raw.shape)
```

```
Crime loaded: (259008, 22)
Education loaded: (1336, 53)
```

```
Unemployment loaded: (1336, 283)
Income loaded: (1336, 5)
Population loaded: (1336, 5)
Tract loaded: (3265, 14)
```

Cleaning data

Cleaning crime data

```python
# Clean crime data
crime = crime_raw[[
    "Date",
    "Primary Type",
    "Year",
    "Latitude",
    "Longitude"
]].copy()

# Set 31 "Primary Type" to 4 types
violent = {
    "HOMICIDE",
    "BATTERY",
    "ASSAULT",
    "ROBBERY",
    "CRIMINAL SEXUAL ASSAULT",
    "SEX OFFENSE",
    "KIDNAPPING",
    "STALKING",
    "INTIMIDATION"
}

property_crime = {
    "THEFT",
    "BURGLARY",
    "MOTOR VEHICLE THEFT",
    "CRIMINAL DAMAGE",
    "CRIMINAL TRESPASS",
    "ARSON",
    "DECEPTIVE PRACTICE"
}
```

```python
regulatory = {
    "NARCOTICS",
    "OTHER NARCOTIC VIOLATION",
    "LIQUOR LAW VIOLATION",
    "WEAPONS VIOLATION",
    "CONCEALED CARRY LICENSE VIOLATION",
    "GAMBLING",
    "PROSTITUTION",
    "PUBLIC INDECENCY",
    "OBSCENITY",
    "HUMAN TRAFFICKING"
}

other = {
    "OTHER OFFENSE",
    "OFFENSE INVOLVING CHILDREN",
    "INTERFERENCE WITH PUBLIC OFFICER",
    "PUBLIC PEACE VIOLATION",
    "NON-CRIMINAL"
}

# Function to classify crime
def classify_crime(x):
    if x in violent:
        return "Violent"
    elif x in property_crime:
        return "Property"
    elif x in regulatory:
        return "Regulatory"
    else:
        return "Other"

# Create new column and drop "Primary Type" column
crime["crime_type"] = crime["Primary Type"].apply(classify_crime)

crime = crime.drop(columns="Primary Type")
```

Cleaning education data

```python
# Remove the first row (metadata row)
edu = edu_raw.iloc[3:][[
    "GEO_ID",
    "B15003_001E",
    "B15003_021E",
    "B15003_022E",
    "B15003_023E",
    "B15003_024E"
]].copy()

# Extract tract_id
edu["tract_id"] = edu["GEO_ID"].str[-11:]

# Convert to numeric
cols = [
    "B15003_001E",
    "B15003_021E",
    "B15003_022E",
    "B15003_023E",
    "B15003_024E"
]

edu[cols] = edu[cols].apply(pd.to_numeric, errors="coerce")

# Bachelor+ population
edu["bachelor_plus"] = (
    edu["B15003_021E"] +
    edu["B15003_022E"] +
    edu["B15003_023E"] +
    edu["B15003_024E"]
)

# Bachelor+ rate
edu["edu_rate"] = 100 * edu["bachelor_plus"] / edu["B15003_001E"]

# Keep only needed columns
edu = edu[["tract_id", "edu_rate"]]

print("Education cleaned:", edu.shape)
display("Education table:", edu.head())
```

```
Education cleaned: (1333, 2)
```

```
'Education table:'
```

	tract_id	edu_rate
3	17031010100	53.392237
4	17031010201	36.340316
5	17031010202	48.261965
6	17031010300	52.186047
7	17031010400	62.710595

Cleaning unemployment data

```python
# Clean unemployment data
unemp = unemp_raw.iloc[3:][[
    "GEO_ID",
    "S2301_C04_001E"
]].copy()

# Extract tract id
unemp["tract_id"] = unemp["GEO_ID"].str[-11:]

# Convert to numeric
unemp["S2301_C04_001E"] = pd.to_numeric(
    unemp["S2301_C04_001E"],
    errors="coerce"
)

# Rename column
unemp = unemp.rename(
    columns={"S2301_C04_001E": "unemployment_rate"}
)

# Keep final columns
unemp = unemp[[
    "tract_id",
    "unemployment_rate"
]]
```

```
print("Unemployment cleaned:", unemp.shape)
display("Unemployment table:", unemp.head())
```

```
Unemployment cleaned: (1333, 2)
```

```
'Unemployment table:'
```

	tract_id	unemployment_rate
3	17031010100	6.2
4	17031010201	11.0
5	17031010202	10.5
6	17031010300	6.0
7	17031010400	3.1

Cleaning income data

```
# Clean income data
income = income_raw.iloc[3:][[
    "GEO_ID",
    "B19013_001E"
]].copy()

# Extract tract id
income["tract_id"] = income["GEO_ID"].str[-11:]

# Convert to numeric
income["B19013_001E"] = pd.to_numeric(
    income["B19013_001E"],
    errors="coerce"
)

# Rename column
income = income.rename(
    columns={"B19013_001E": "median_income"}
)

# Keep final columns
income = income[[
```

```
    "tract_id",
    "median_income"
]]

print("Income cleaned:", income.shape)
display("Income table:", income.head())
```

```
Income cleaned: (1333, 2)
```

```
'Income table:'
```

	tract_id	median_income
3	17031010100	74145.0
4	17031010201	63056.0
5	17031010202	53220.0
6	17031010300	73105.0
7	17031010400	54083.0

Cleaning population data

```
# Clean population data
pop = pop_raw.iloc[3:][[
    "GEO_ID",
    "B01003_001E"
]].copy()

# Extract tract id
pop["tract_id"] = pop["GEO_ID"].str[-11:]

# Convert to numeric
pop["B01003_001E"] = pd.to_numeric(
    pop["B01003_001E"],
    errors="coerce"
)

# Rename column
pop = pop.rename(
    columns={"B01003_001E": "population"}
```

```
)

# Keep final columns
pop = pop[[
    "tract_id",
    "population"
]]

print("Population cleaned:", pop.shape)
display("Population table:", pop.head())
```

```
Population cleaned: (1333, 2)
```

```
'Population table:'
```

	tract_id	population
3	17031010100	4198
4	17031010201	7361
5	17031010202	2482
6	17031010300	6477
7	17031010400	4545

Subset data for only Chicago tracts

Tracts Geodata: keep only Chicago tract IDs

```
# Load tract and Chicago boundary shapefiles
tracts = gpd.read_file(DATA_DIR / "tl_2023_17_tract" / "tl_2023_17_tract.shp")
chicago = gpd.read_file(DATA_DIR / "Boundaries - City_20260303" / "geo_export_06d21f1c-6fda-4

# Align CRS
chicago = chicago.to_crs(tracts.crs)

# Keep tracts that intersect Chicago boundary (without cutting geometry)
tracts_chi = gpd.sjoin(
    tracts,
    chicago[["geometry"]],
    how="inner",
```

```
    predicate="intersects"
).drop(columns=["index_right"])

# Clip tracts to Chicago boundary
tracts_chi = gpd.clip(tracts_chi, chicago)

# Ensure tract id format
tracts_chi["GEOID"] = tracts_chi["GEOID"].astype(str).str.zfill(11)

# Chicago tract id set
chi_tract_ids = set(tracts_chi["GEOID"])
print(f"Total tracts in Chicago: {len(chi_tract_ids)}")
```

```
Total tracts in Chicago: 862
```

Crime Geodata: add tract_id column

```
# Make crime GeoDataFrame (Chicago only)
crime_geo = crime.dropna(subset=["Latitude", "Longitude"]).copy()

crime_gdf = gpd.GeoDataFrame(
    crime_geo,
    geometry=gpd.points_from_xy(crime_geo["Longitude"], crime_geo["Latitude"]),
    crs="EPSG:4326"
)

tracts_chi_4326 = tracts_chi.to_crs("EPSG:4326")

crime_tract = gpd.sjoin(
    crime_gdf,
    tracts_chi_4326[["GEOID", "geometry"]],
    how="inner",
    predicate="within"
).rename(columns={"GEOID": "tract_id"})

crime_tract["tract_id"] = crime_tract["tract_id"].astype(str).str.zfill(11)

print(crime_tract.shape)
display(crime_tract.head())
```

```
(256652, 8)
```

	Date	Year	Latitude	Longitude	crime_type	geometry	in
0	12/31/2024 11:58:00 PM	2024	41.771470	-87.590742	Property	POINT (-87.59074 41.77147)	20
1	12/31/2024 11:56:00 PM	2024	41.808501	-87.616563	Property	POINT (-87.61656 41.8085)	25
2	12/31/2024 11:55:00 PM	2024	41.752749	-87.741498	Violent	POINT (-87.7415 41.75275)	11
3	12/31/2024 11:53:00 PM	2024	41.774269	-87.599710	Property	POINT (-87.59971 41.77427)	69
4	12/31/2024 11:50:00 PM	2024	41.902858	-87.770537	Violent	POINT (-87.77054 41.90286)	88

Education Geodata: keep only Chicago tracts

```
edu_gdf = tracts_chi.merge(
    edu,
    left_on="GEOID",
    right_on="tract_id",
    how="left"
)

edu_gdf.head()
```

	STATEFP	COUNTYFP	TRACTCE	GEOID	GEOIDFQ	NAME	NAMELSAD
0	17	031	826500	17031826500	1400000US17031826500	8265	Census Tract
1	17	031	530502	17031530502	1400000US17031530502	5305.02	Census Tract
2	17	031	530600	17031530600	1400000US17031530600	5306	Census Tract
3	17	031	530100	17031530100	1400000US17031530100	5301	Census Tract
4	17	031	491300	17031491300	1400000US17031491300	4913	Census Tract

Unemployment Geodata: keep only Chicago tracts

```
unemp_gdf = tracts_chi.merge(
    unemp,
    left_on="GEOID",
    right_on="tract_id",
    how="left"
)

unemp_gdf.head()
```

	STATEFP	COUNTYFP	TRACTCE	GEOID	GEOIDFQ	NAME	NAMELSAD
0	17	031	826500	17031826500	1400000US17031826500	8265	Census Tract
1	17	031	530502	17031530502	1400000US17031530502	5305.02	Census Tract
2	17	031	530600	17031530600	1400000US17031530600	5306	Census Tract
3	17	031	530100	17031530100	1400000US17031530100	5301	Census Tract
4	17	031	491300	17031491300	1400000US17031491300	4913	Census Tract

Income Geodata: keep only Chicago tracts

```
income_gdf = tracts_chi.merge(
    income,
    left_on="GEOID",
    right_on="tract_id",
    how="left"
)

income_gdf.head()
```

	STATEFP	COUNTYFP	TRACTCE	GEOID	GEOIDFQ	NAME	NAMELSAD
0	17	031	826500	17031826500	1400000US17031826500	8265	Census Tract
1	17	031	530502	17031530502	1400000US17031530502	5305.02	Census Tract
2	17	031	530600	17031530600	1400000US17031530600	5306	Census Tract
3	17	031	530100	17031530100	1400000US17031530100	5301	Census Tract
4	17	031	491300	17031491300	1400000US17031491300	4913	Census Tract

Population Geodata: keep only Chicago tracts

```
pop_gdf = tracts_chi.merge(
    pop,
    left_on="GEOID",
    right_on="tract_id",
    how="left"
)

pop_gdf.head()
```

	STATEFP	COUNTYFP	TRACTCE	GEOID	GEOIDFQ	NAME	NAMELSAD
0	17	031	826500	17031826500	1400000US17031826500	8265	Census Tract
1	17	031	530502	17031530502	1400000US17031530502	5305.02	Census Tract
2	17	031	530600	17031530600	1400000US17031530600	5306	Census Tract
3	17	031	530100	17031530100	1400000US17031530100	5301	Census Tract
4	17	031	491300	17031491300	1400000US17031491300	4913	Census Tract

Merge Crime and ACS variables into tract GeoDataFrame

```
# Aggregate crime_count by tract_id
crime_by_tract = (
    crime_tract
    .groupby("tract_id", as_index=False)
    .size()
    .rename(columns={"size": "crime_count"})
)

# Merge crime + ACS into Chicago tracts GeoDataFrame
tracts_analysis = (
    tracts_chi
    .merge(pop, left_on="GEOID", right_on="tract_id", how="left")
    .drop(columns=["tract_id"])
    .merge(crime_by_tract, left_on="GEOID", right_on="tract_id", how="left")
    .drop(columns=["tract_id"])
    .merge(edu, left_on="GEOID", right_on="tract_id", how="left").drop(columns=["tract_id"])
    .merge(unemp, left_on="GEOID", right_on="tract_id", how="left").drop(columns=["tract_id"
    .merge(income, left_on="GEOID", right_on="tract_id", how="left").drop(columns=["tract_id
)

# Fill tracts with no crimes + compute rate (per 1000 residents)
tracts_analysis["crime_count"] = tracts_analysis["crime_count"].fillna(0).astype(int)
tracts_analysis["crime_rate"] = (tracts_analysis["crime_count"] / tracts_analysis["populatio

tracts_analysis.head()
```

	STATEFP	COUNTYFP	TRACTCE	GEOID	GEOIDFQ	NAME	NAMELSAD
0	17	031	826500	17031826500	1400000US17031826500	8265	Census Tract
1	17	031	530502	17031530502	1400000US17031530502	5305.02	Census Tract

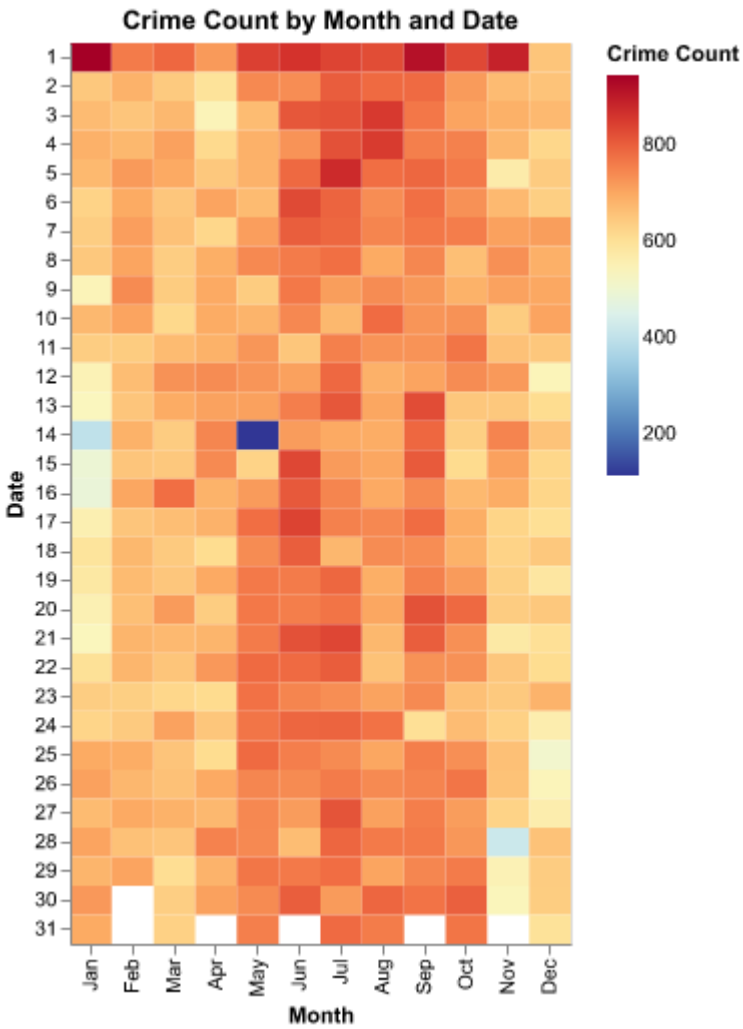
	STATEFP	COUNTYFP	TRACTCE	GEOID	GEOIDFQ	NAME	NAMELSAD
2	17	031	530600	17031530600	1400000US17031530600	5306	Census Tract
3	17	031	530100	17031530100	1400000US17031530100	5301	Census Tract
4	17	031	491300	17031491300	1400000US17031491300	4913	Census Tract

Part 1: Crime count / rate patterns amoung Tracts

Static plot 1: Crime Count by Month and Date

```
# Prepare date variables
crime_time = crime_tract.copy()
crime_time["Date"] = pd.to_datetime(crime_time["Date"], errors="coerce")

crime_time["month"] = crime_time["Date"].dt.month
crime_time["date"] = crime_time["Date"].dt.day

# Aggregate crime count
crime_calendar = (
    crime_time
    .groupby(["month", "date"], as_index=False)
    .size()
    .rename(columns={"size": "crime_count"})
)

# Month labels
month_order = [
    "Jan","Feb","Mar","Apr","May","Jun",
    "Jul","Aug","Sep","Oct","Nov","Dec"
]

crime_calendar["month_name"] = crime_calendar["month"].apply(lambda x: month_order[x-1])

# Heatmap
calendar_heatmap = (
    alt.Chart(crime_calendar)
    .mark_rect()
    .encode(
        x=alt.X(
            "month_name:N",
```

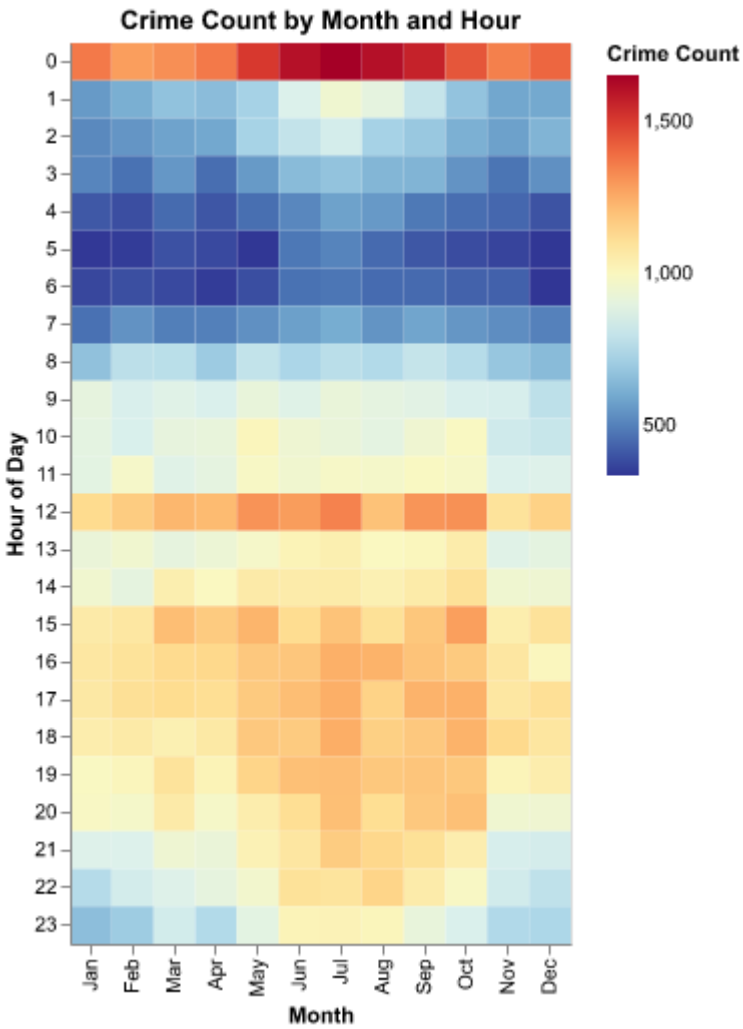
```
            title="Month",
            sort=month_order
        ),
        y=alt.Y(
            "date:O",
            title="Date"
        ),
        color=alt.Color(
            "crime_count:Q",
            title="Crime Count",
            scale=alt.Scale(scheme="redyellowblue", reverse=True)
        ),
        tooltip=[
            alt.Tooltip("month_name:N", title="Month"),
            alt.Tooltip("date:O", title="Date"),
            alt.Tooltip("crime_count:Q", title="Crime Count")
        ]
    )
    .properties(
        width=250,
        height=450,
        title="Crime Count by Month and Date"
    )
)

calendar_heatmap
```



Static plot 2: Crime Count by Month and Hour

```
# Static plot 2: Crime count by Month and Hour

crime_time = crime_tract.copy()
crime_time["Date"] = pd.to_datetime(crime_time["Date"], errors="coerce")

crime_time["month"] = crime_time["Date"].dt.month
crime_time["hour"] = crime_time["Date"].dt.hour

# Aggregate counts
```

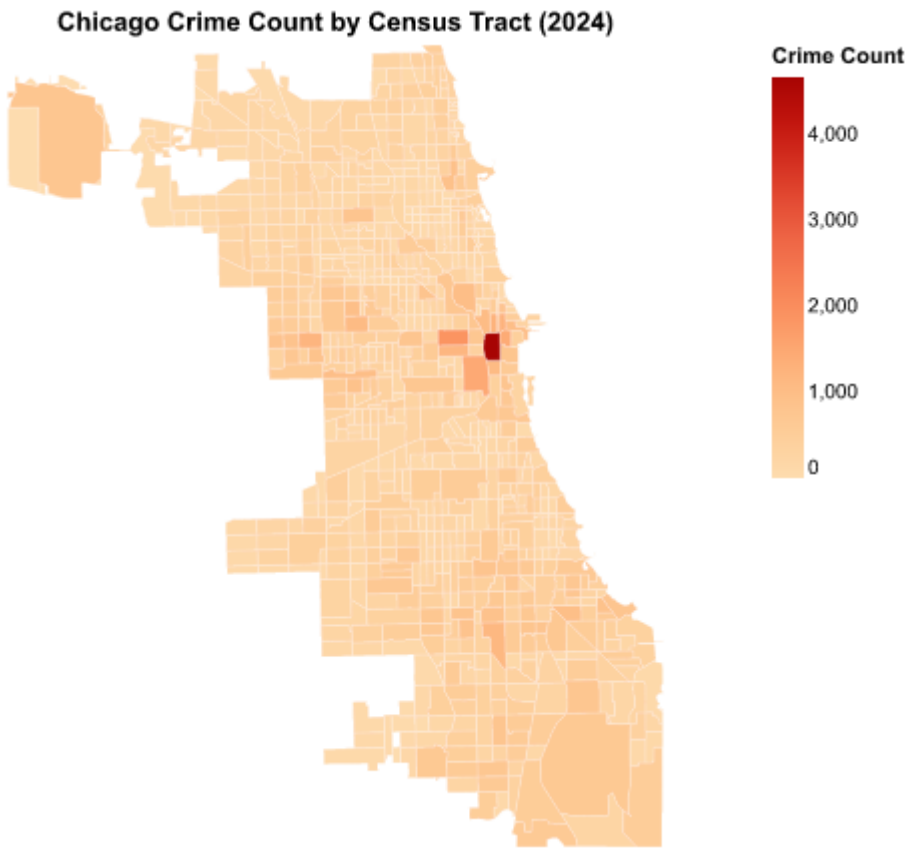
```python
crime_month_hour = (
    crime_time
    .groupby(["month", "hour"], as_index=False)
    .size()
    .rename(columns={"size": "crime_count"})
)

# Month labels
month_order = ["Jan","Feb","Mar","Apr","May","Jun",
               "Jul","Aug","Sep","Oct","Nov","Dec"]

crime_month_hour["month_name"] = crime_month_hour["month"].apply(lambda x: month_order[x-1])

# Heatmap
crime_hour_heatmap = (
    alt.Chart(crime_month_hour)
    .mark_rect()
    .encode(
        x=alt.X("month_name:N", title="Month", sort=month_order),
        y=alt.Y("hour:O", title="Hour of Day"),
        color=alt.Color(
            "crime_count:Q",
            title="Crime Count",
            scale=alt.Scale(scheme="redyellowblue", reverse=True)
        ),
        tooltip=[
            alt.Tooltip("month_name:N", title="Month"),
            alt.Tooltip("hour:O", title="Hour"),
            alt.Tooltip("crime_count:Q", title="Crime Count")
        ]
    )
    .properties(
        width=250,
        height=450,
        title="Crime Count by Month and Hour"
    )
)

crime_hour_heatmap
```



Static plot 3: Crime Count Choropleth Map

```
# Crime count choropleth map (Chicago tracts)

tracts_plot = tracts_analysis.to_crs("EPSG:4326").copy()
geojson_dict = json.loads(tracts_plot.to_json())

crime_count_map = (
    alt.Chart(
        alt.InlineData(
            values=geojson_dict,
```

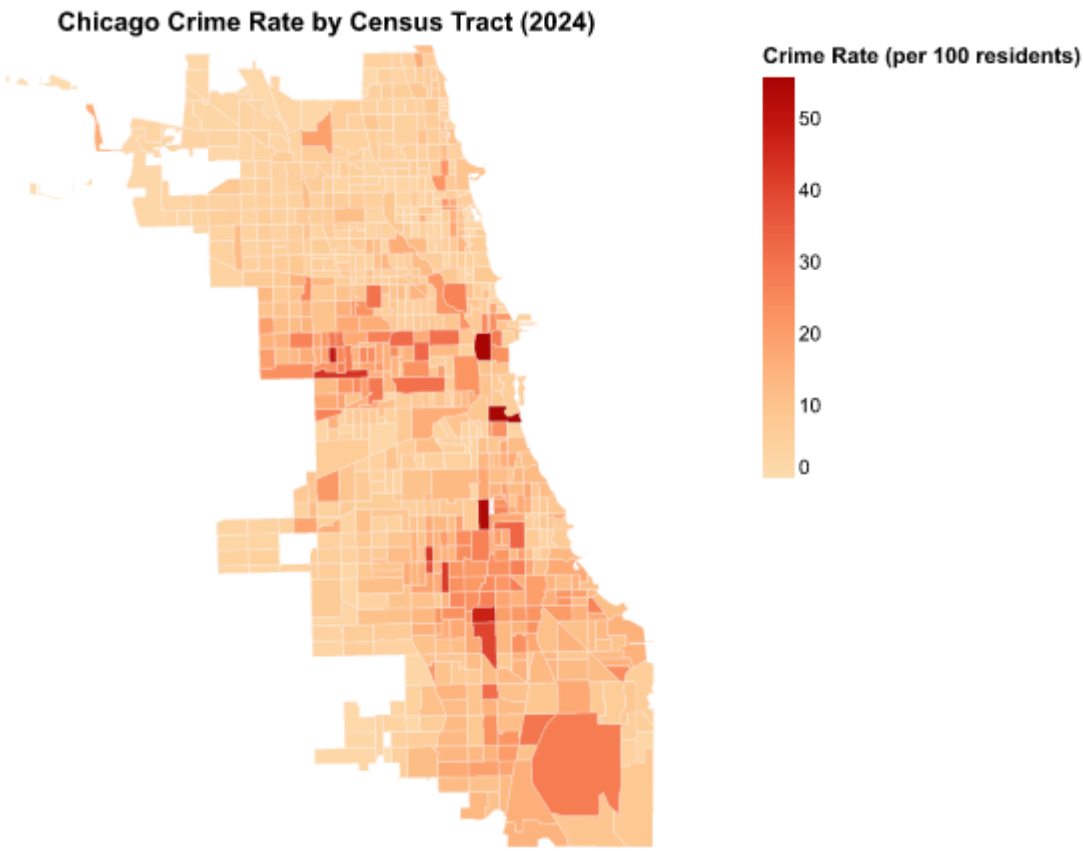
```
            format=alt.DataFormat(type="json", property="features")
        )
    )
    .mark_geoshape(stroke="white", strokeWidth=0.2)
    .encode(
        color=alt.Color(
            "properties.crime_count:Q",
            title="Crime Count",
            scale=alt.Scale(scheme="orangered")
        ),
        tooltip=[
            alt.Tooltip("properties.GEOID:N", title="Tract"),
            alt.Tooltip("properties.crime_count:Q", title="Crime count"),
        ],
    )
    .properties(
        width=400,
        height=400,
        title="Chicago Crime Count by Census Tract (2024)"
    )
    .project(type="mercator")
)

crime_count_map
```



Static plot 4: Crime Rate Choropleth Map

```
# Crime rate choropleth map (per 100 residents)

tracts_plot = tracts_analysis.to_crs("EPSG:4326").copy()
geojson_dict = json.loads(tracts_plot.to_json())

crime_rate_map = (
    alt.Chart(
        alt.InlineData(
            values=geojson_dict,
            format=alt.DataFormat(type="json", property="features")
        )
    )
    .mark_geoshape(stroke="white", strokeWidth=0.2)
    .encode(
```

```
        color=alt.Color(
            "properties.crime_rate:Q",
            title="Crime Rate (per 100 residents)",
            scale=alt.Scale(scheme="orangered")
        ),
        tooltip=[
            alt.Tooltip("properties.GEOID:N", title="Tract"),
            alt.Tooltip("properties.crime_count:Q", title="Crime count"),
            alt.Tooltip("properties.population:Q", title="Population"),
            alt.Tooltip("properties.crime_rate:Q", title="Crime rate", format=".2f"),
        ],
    )
    .properties(
        width=400,
        height=400,
        title="Chicago Crime Rate by Census Tract (2024)"
    )
    .project(type="mercator")
)

crime_rate_map
```

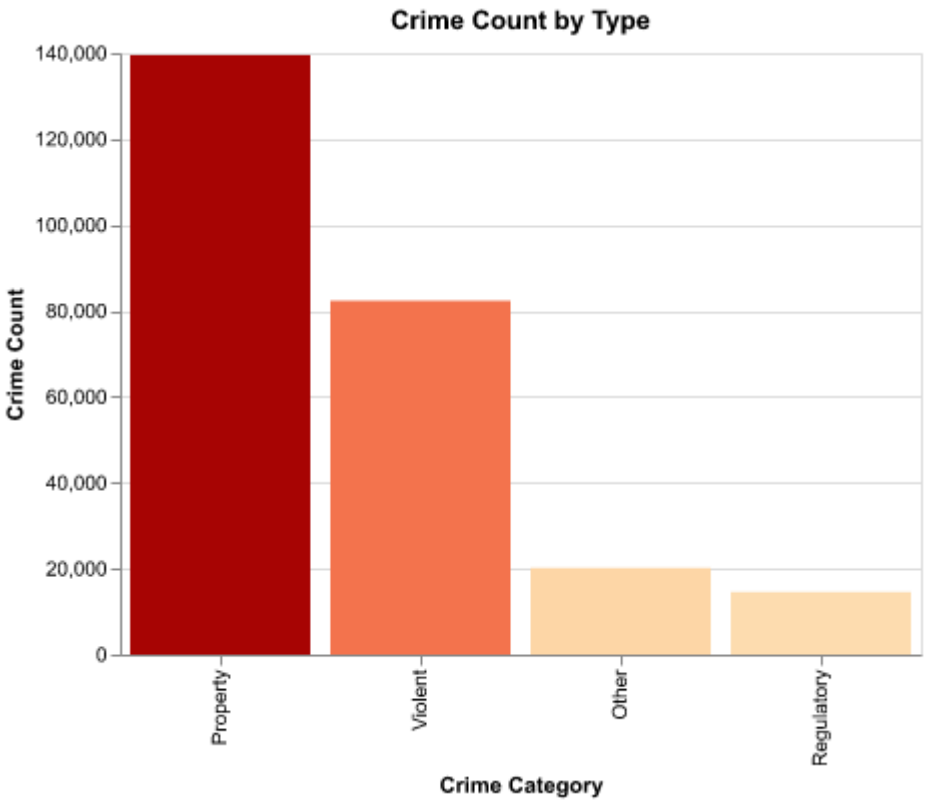


Part 2:

Static plot 5: Crime Count by Type

```
# Aggregate crime counts by grouped crime category
crime_type_counts = (
    crime_tract
    .groupby("crime_type", as_index=False)
    .size()
    .rename(columns={"size": "crime_count"})
)

# Create bar chart of crime counts by type
crime_type_bar = (
    alt.Chart(crime_type_counts)
```

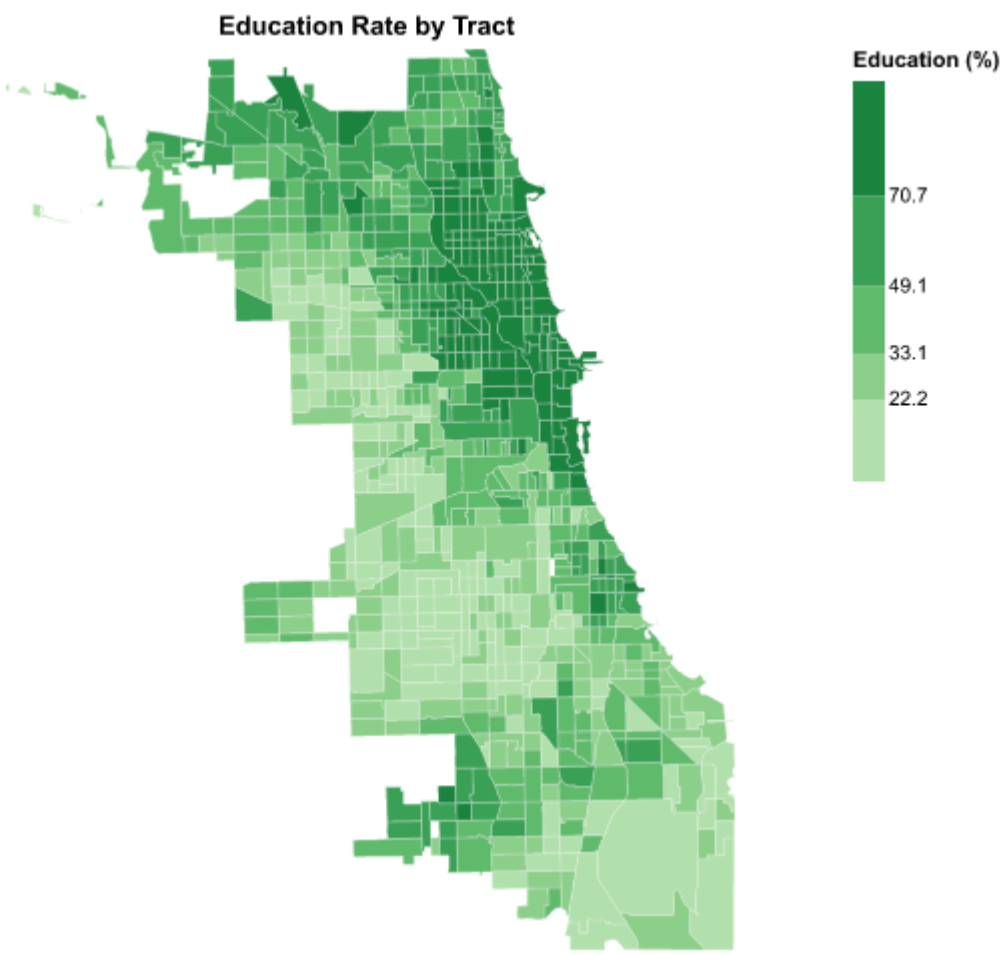
```
    .mark_bar()
    .encode(
        x=alt.X("crime_type:N", title="Crime Category", sort="-y"),
        y=alt.Y("crime_count:Q", title="Crime Count"),
        color=alt.Color(
            "crime_count:Q",
            scale=alt.Scale(scheme="orangered"),
            legend=None
        ),
        tooltip=[
            "crime_type:N",
            "crime_count:Q"
        ]
    )
    .properties(
        width=400,
        height=300,
        title="Crime Count by Type"
    )
)

crime_type_bar
```



Static plot 6: Choropleths for Education Rate

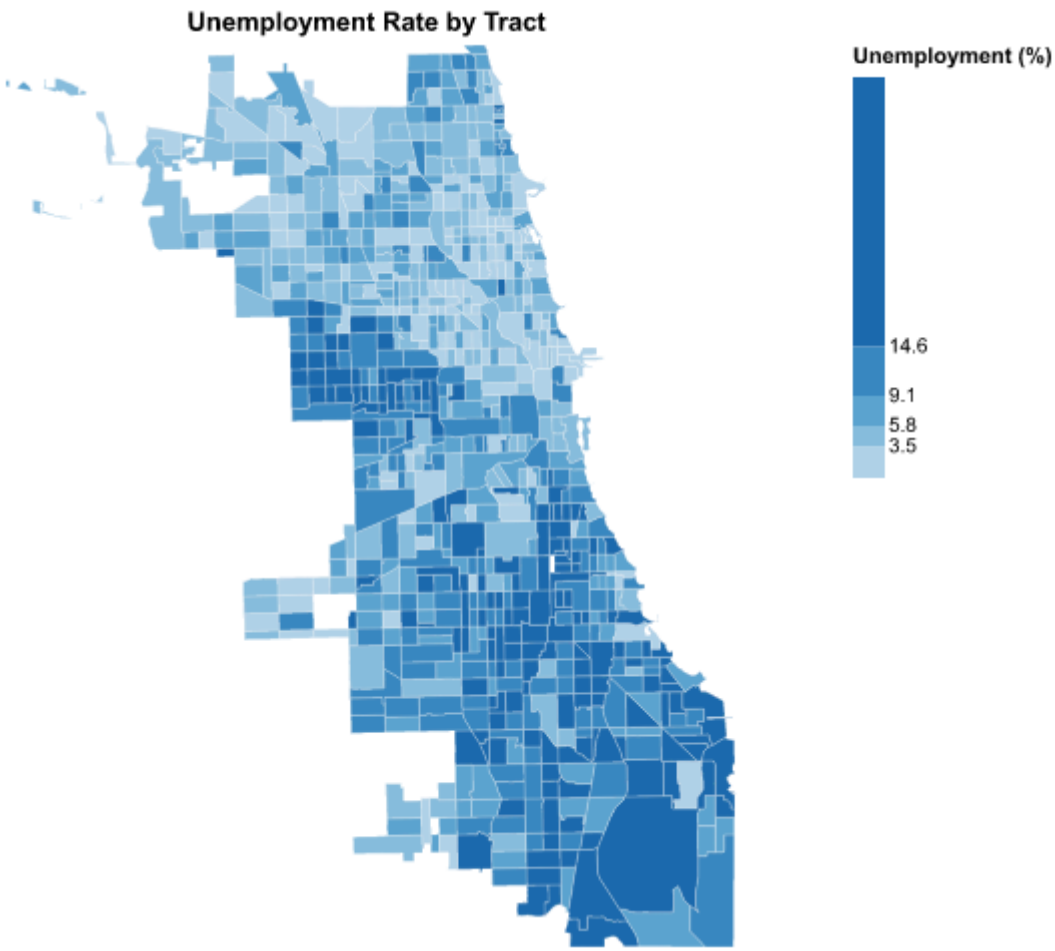
```
# Set base map
tracts_plot = tracts_analysis.to_crs("EPSG:4326").copy()
geojson_dict = json.loads(tracts_plot.to_json())

base = (
    alt.Chart(
        alt.InlineData(
            values=geojson_dict,
            format=alt.DataFormat(type="json", property="features")
        )
    )
    .mark_geoshape(stroke="white", strokeWidth=0.2)
    .project(type="mercator")
    .properties(width=450, height=450)
)
```

```
# Education choropleth
edu_map = base.encode(
    color=alt.Color(
        "properties.edu_rate:Q",
        title="Education (%)",
        scale=alt.Scale(scheme="greens", type="quantile")
    ),
    tooltip=[
        alt.Tooltip("properties.GEOID:N", title="Tract"),
        alt.Tooltip("properties.edu_rate:Q", title="Education (%)", format=".1f")
    ]
).properties(title="Education Rate by Tract")

edu_map
```

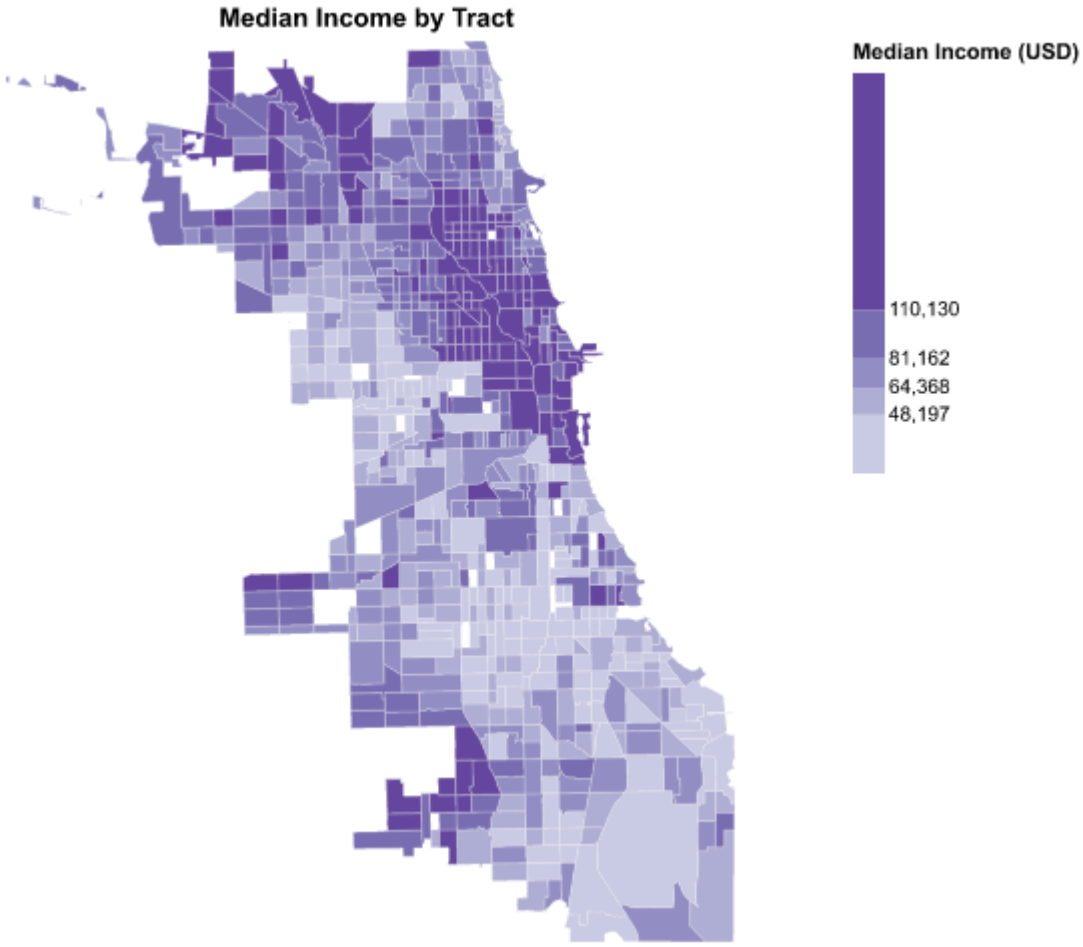


Static plot 7: Choropleths for Unemployment Rate

```
# Unemployment choropleth
unemp_map = base.encode(
    color=alt.Color(
        "properties.unemployment_rate:Q",
        title="Unemployment (%)",
        scale=alt.Scale(scheme="blues", type="quantile")
    ),
    tooltip=[
        alt.Tooltip("properties.GEOID:N", title="Tract"),
        alt.Tooltip("properties.unemployment_rate:Q", title="Unemployment (%)", format=".1f")
    ]
).properties(title="Unemployment Rate by Tract")

unemp_map
```



Static plot 8: Choropleths for Median Income

```
# Income choropleth
income_map = base.encode(
    color=alt.Color(
        "properties.median_income:Q",
        title="Median Income (USD)",
        scale=alt.Scale(scheme="purples", type="quantile")
    ),
    tooltip=[
        alt.Tooltip("properties.GEOID:N", title="Tract"),
        alt.Tooltip("properties.median_income:Q", title="Median Income", format=",.0f")
    ]
```

```
).properties(title="Median Income by Tract")

income_map
```